

Resumo P2

side 14- Padrões de projeto

- Os padrões de projeto são técnicas para resolver problemas comuns em design de software
- Eles fornecem uma solução esquematizada que ajuda a resolver problemas de design de software de maneira eficiente e eficaz.

Três categorias principais: Criacionais, Estruturais e Comportamentais.

Criacionais	Estruturais	Comportamentais
Abstract Factory Factory Method Singleton Builder Prototype	Proxy Adapter Facade Decorator Bridge Composite Flyweight	Strategy Observer Template Method Visitor Chain of Responsibility Command Interpreter Iterator Mediator Memento State

(01) Fábrica (Criacional)

- É usado para criar uma instância de várias classes derivadas
- Por exemplo, uma fábrica de conexões de rede poderia retornar uma conexão TCP ou UDP dependendo de parâmetros de entrada. Centraliza a criação de objetos, facilitando mudanças no processo de instânciação.

```
class ChannelFactory {  
    public static Channel create() { // método fábrica estático  
        return new TCPChannel();  
    }  
}
```

Único ponto de mudança se quisermos usar UDP

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

⇒

Com Método Fábrica Estático

```
void f() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c = ChannelFactory.create();  
    ...  
}
```

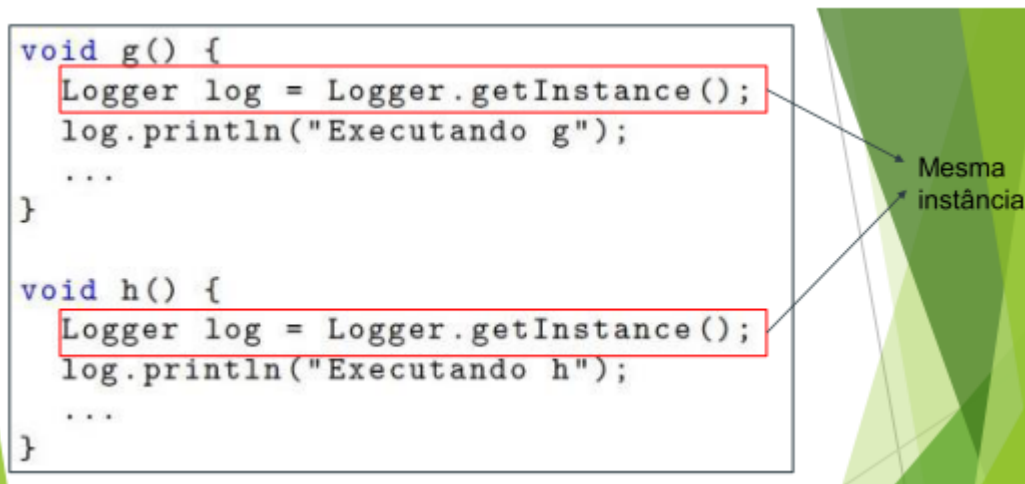
(02) Singleton (Criacional)

- Garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a essa instância

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

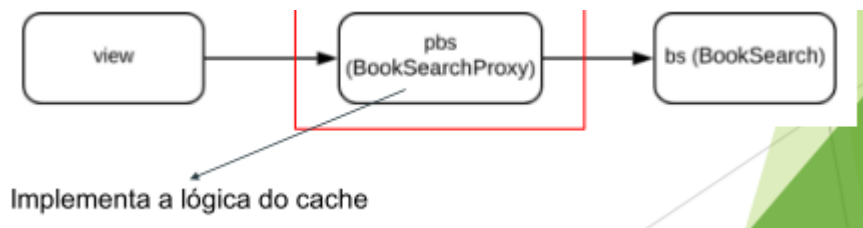
Problema:

Cada método cliente usa sua própria instância de Logger



(03) Proxy (Estrutural)

- Fornece um substituto ou representante de outro objeto para controlar o acesso a ele
- Usado frequentemente em situações onde objetos precisam de um intermediário ou quando funcionalidades adicionais são necessárias quando acessando um objeto particular



(04) Adaptador (Estrutural)

- Converter a interface de uma classe em outra interface que os clientes esperam
- Permitem que classes trabalhem juntas, apesar de interfaces incompatíveis

(05) Fachada (Estrutural)

- Fornece uma interface unificada para um conjunto de interfaces em um subsistema
- Define uma interface de nível mais alto que torna o subsistema mais fácil de ser usado

(06) Decorador (Estrutural)

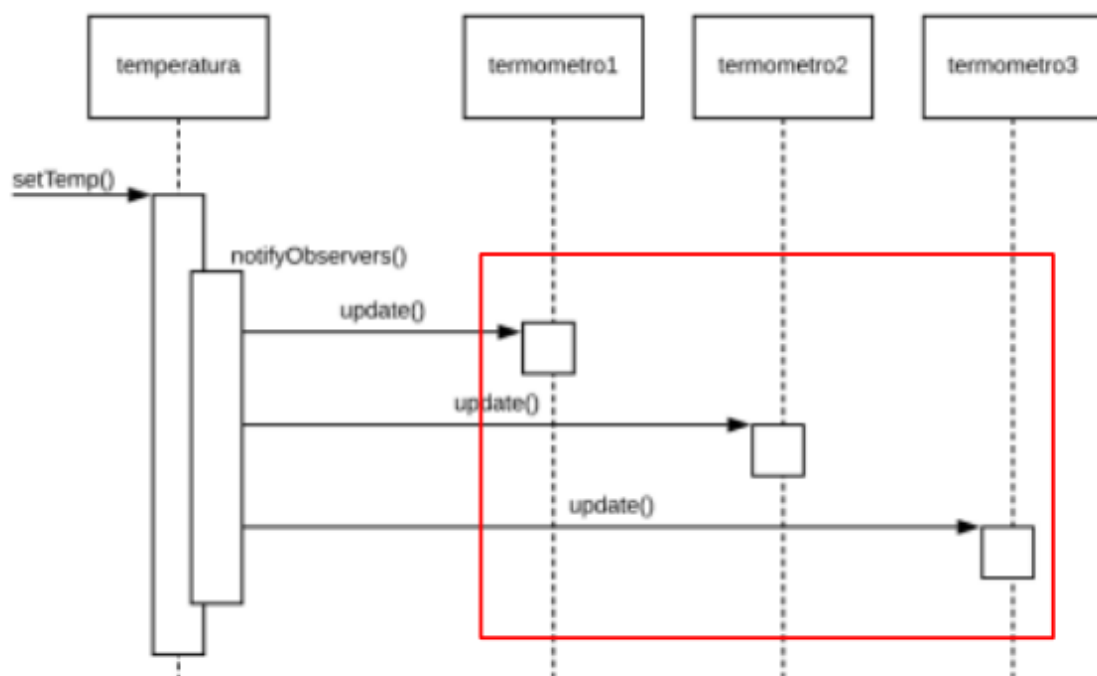
- Adiciona responsabilidades adicionais a um objeto dinamicamente
- Fornecem uma alternativa flexível ao uso de subclasses para extensão de funcionalidades
- Por meio da composição e não mais por meio de herança

(07) Strategy(Comportamentais)

- Parametrizar os algoritmos usado por uma classe
- Tornar uma classe “aberta” a novos algoritmos

(08)Obeservador(Comportamentais)

- Um modo de notificação de mudança a vários objetos para garantir a consistência entre classes estreitamente cooperativas
- É frequentemente usado em interfaces gráficas para informar e atualizar automaticamente outras partes de um aplicativo quando um objeto muda



(09)Template Method(Comportamentais)

- Define o esqueleto de um algoritmo em uma operação, postergando alguns passos para subclasses
- Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar a estrutura do algoritmo

Quando não usar padrões de projeto?

- Se a chance de mudanças é zero, não precisamos de padrões de projeto

Slide 15- Arquitetura

Padrões arquiteturais

- “Modelos” pré-definidos para arquiteturas de software
- **Arquitetura monolítica:** Sistema operacional é um único arquivo
 - Gerenci de processos, memória, arquivos
- **Arquitetura Microkernel:** Só possui serviços essenciais, os demais serviços rodam como procesos independentes

ARQUITETURA EM CAMADAS

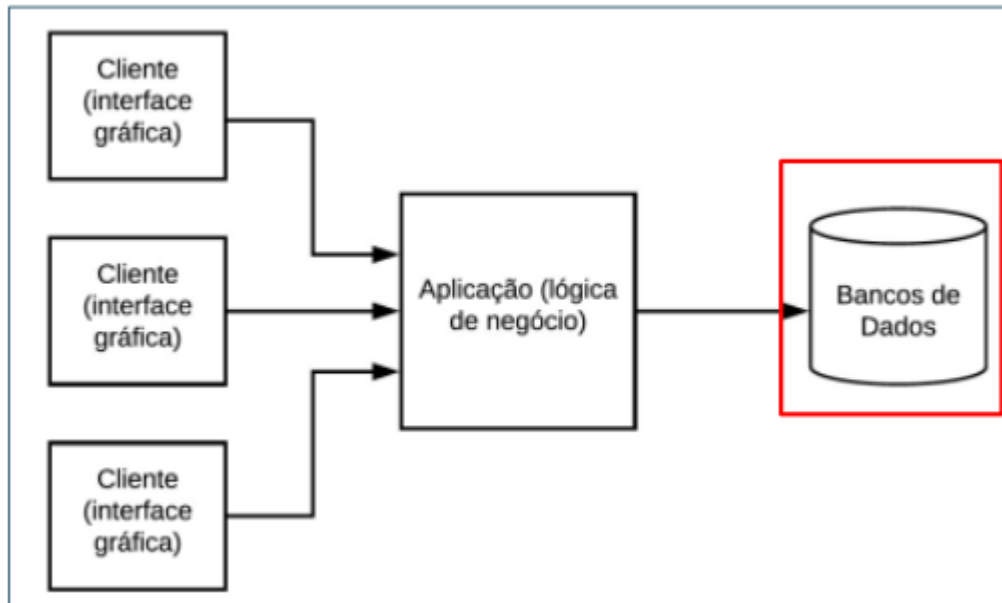
- O sistema é organizado em camadas, de forma hierárquica
- Camada n somente pode usar serviços da camada n-1
- Utilizada em rede computadores e sistemas distribuídos

Vantagens:

- Facilita entendimento
- Facilita troca de uma camada por outra
- Facilita o reúso de uma camada

Variação de três camadas

- Comum em processos de “downsizing” de aplicações corporativas nas décadas de 80 e 90

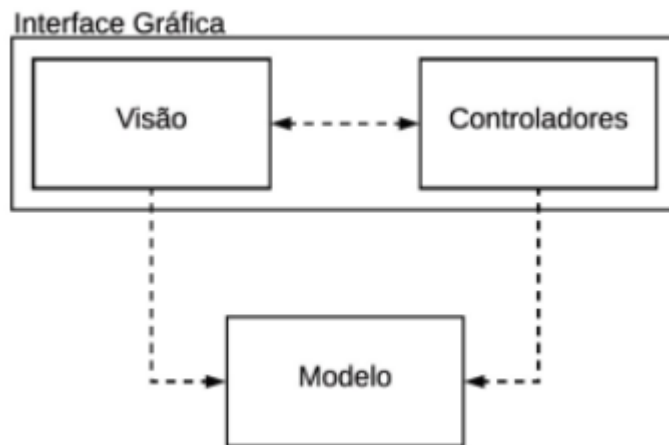


Variação de duas camadas

- Camada 1 (cliente): interface + lógica
- Camada 2 (servidor de BD): banco de dados
 - Desvantagem: todo o processamento é feito no cliente

ARQUITETURA MODEL-VIEW-CONTROLLER (MVC)

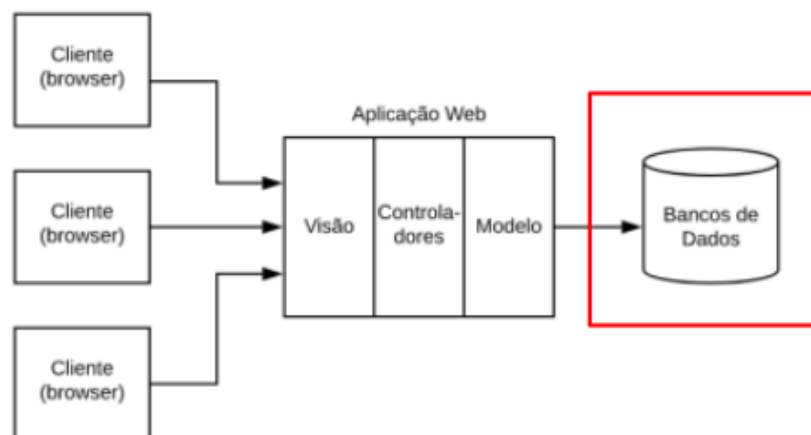
- Dividir as classes de um sistema em 3 grupos:
 - **Visão:** Classe para implementação de GUIs, como janelas, botões, menus, barras de rolagem
 - **Controle:** Classes que tratam eventos produzidos por dispositivos de entrada, como mouse e teclado
 - **Modelo:** classe com lógica de dados da aplicação



- Foi pensando para aplicações desktop “monolíticas” como Microsoft Word e não para aplicações distribuídas

ARQUITETURA MVC WEB

- Adaptação para web, ou seja, para sistemas distribuídos



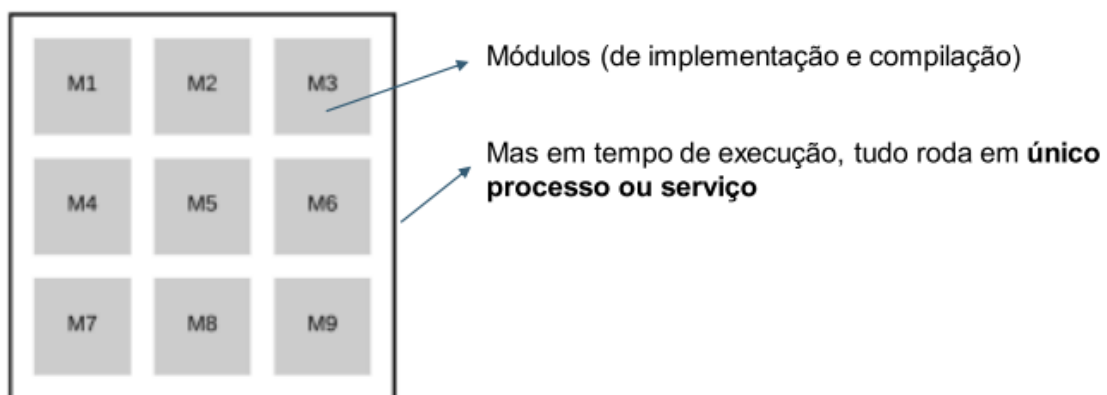
ARQUITETURA SINGLE PAGE (SPAS)

- Funcionam usando um modelo request-response:
 - Cliente requisita uma página ao servidor
 - Servidor envia página e cliente a exibe
 - Cliente solicita no página
- Problema:** interface menos reponsiva e mais lenta

- Podemos ter aplicação que roda no browser, mas que é mais independente do servidor e manipula sua própria interface e armazena os seus dados, mas pode acessar o servidor para buscar mais dados
 - **Exemplo:** GMail, Google Docs, Facebook, Figma, etc
 - Implementados em JavaScript

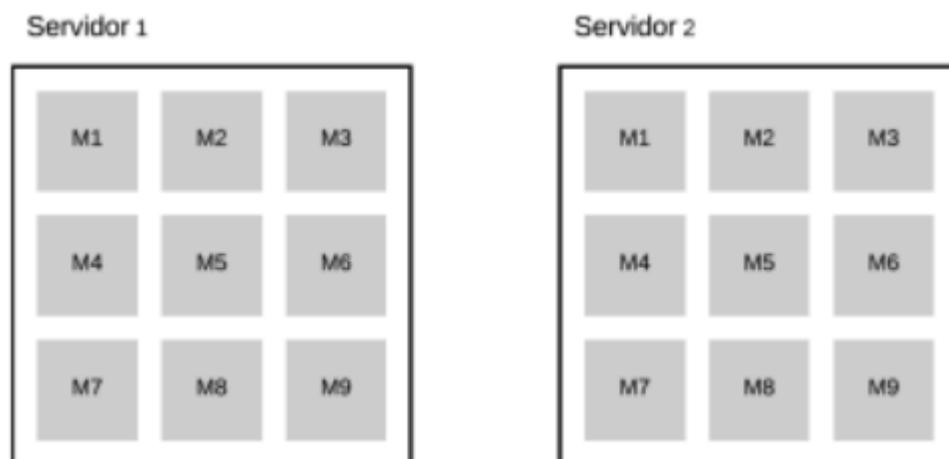
ARQUITETURA BASEADA EM MICROSERVIÇOS

- Monolitos: Em tempo de execução, sistema é um único processo



Problemas

- Deve-se escalar o monolito inteiro, mesmo quando o gargalo de desempenho está em um único módulo

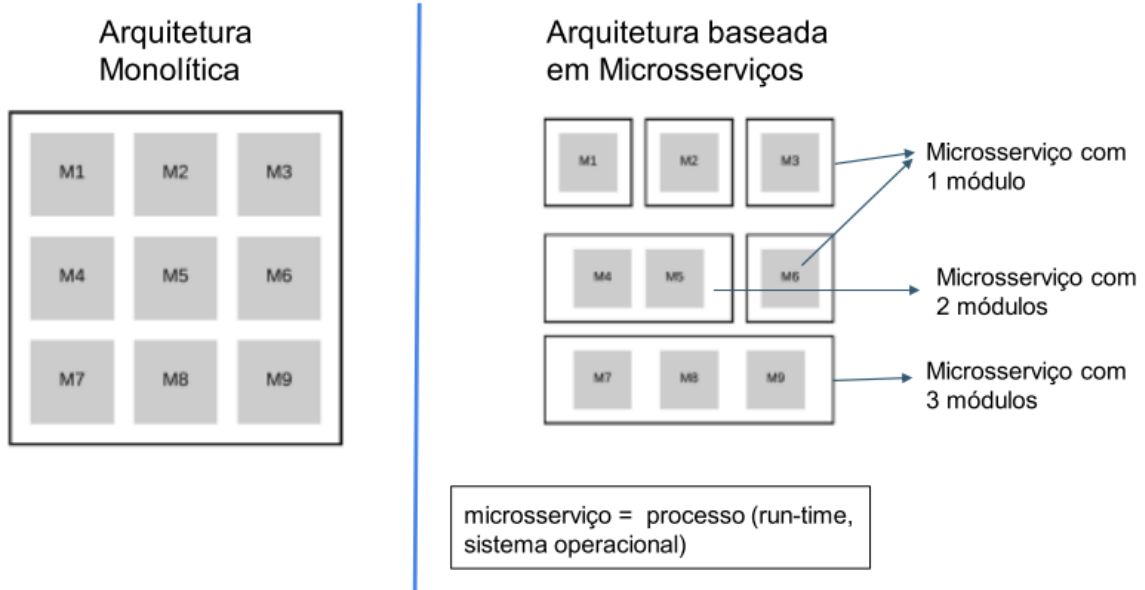


- Processo de release é lento, centralizado e burocrático

- Times não tem poder para colocar módulos em produção, pois mudanças em um módulo podem impactar módulos que já estejam funcionando

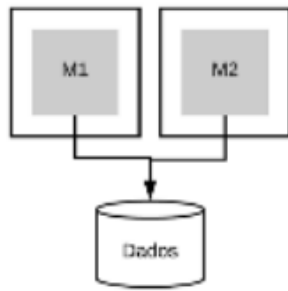
MICROSSERVIÇOS

- Módulos (ou conjunto de módulos) viram processos independentes em tempo de execução

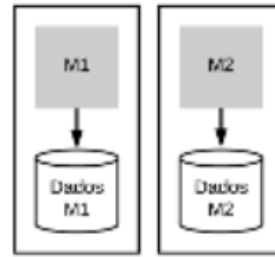


Vantagens

- Pode-se escalar apenas o módulo com problema de desempenho
- Times ganham autonomia para colocar microserviços em produção
- Tecnologias diferentes
- Falhas parciais



Arquitetura que não é recomendada.
Motivo: aumenta acoplamento entre M1 e M2



Arquitetura recomendada. Motivo: não existe acoplamento de dados entre M1 e M2. Logo, M1 e M2 podem evoluir de modo independente.
Se M1 precisar usar serviços de M2 (ou vice-versa), isso deve ocorrer via interfaces

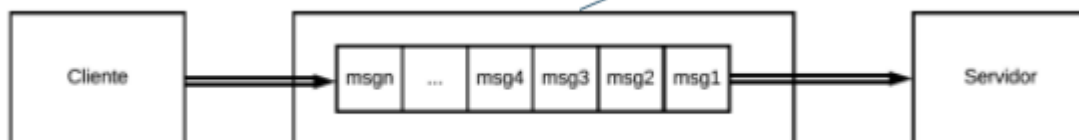
Desvantagens

- Arquitetura é mais complexa:
 - Sistema distribuído
 - Latência
 - Transações distribuídas

Slide 16- Arquitetura

ARQUITETURA ORIENTADA A MENSAGENS

- Arquitetura para aplicações distribuídas
- Clientes não se comunicam diretamente com servidores, mas com um intermediário (fila de mensagens)



Vantagens

- Tolerância a falhas

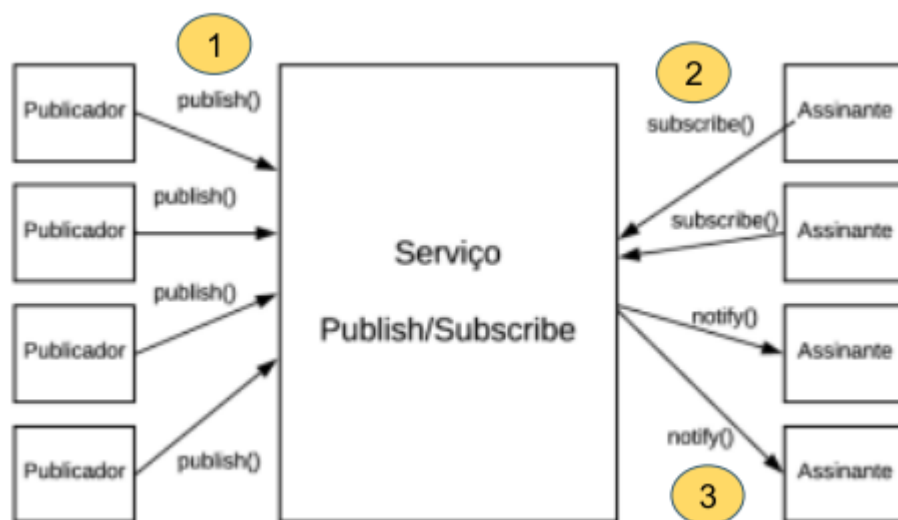
- Mais fácil acrescentar novos servidores (e mais difícil sobrecarregar um servidor com excesso de mensagens)

Comunicação assíncrona

- Cria um acoplamento fraco entre clientes e servidores, pois a comunicação é assíncrona
- Clientes não conhecem servidores e vice-versa
- Clientes e servidores não precisam estar simultaneamente no ar

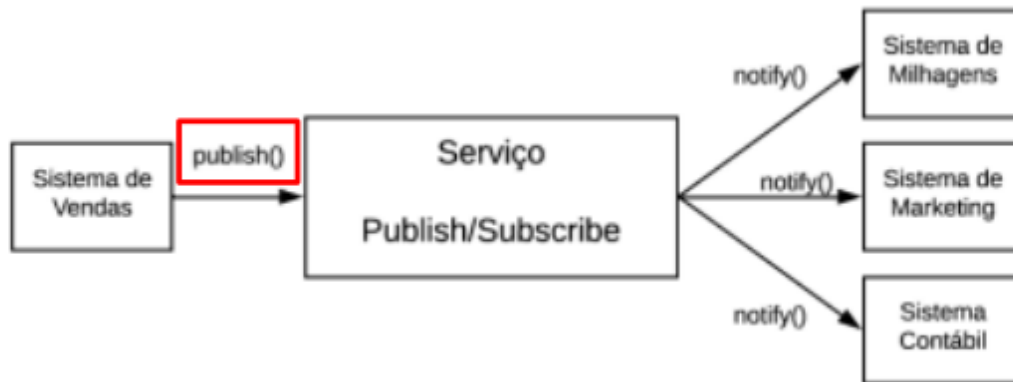
ARQUITETURA PUBLISH/SUBSCRIBE

- Aperfeiçoamento de fila de mensagens
- As mensagens são chamadas de eventos
- Sistemas podem: (1) publicar eventos; (2) assinar eventos; (3) serem notificados sobre a ocorrência de eventos



Exemplo: Sistema de companhia aérea

- Evento: venda de passagem



ARQUITETURA PIPES E FILTROS

- Programas são chamados de filtros e se comunicam por meio de pipes (que agem como buffers)
- Bastante flexível

Exemplo: `ls | grep csv | sort`

Filtro Pipe

ARQUITETURA CLIENTE/SERVIDOR

- Muito comum em serviços básicos de redes
 - Serviços de impressão
 - Serviços de arquivos
 - Serviços Web

ARQUITETURA PEER-TO-PEER

- Todo nodo (ou par) pode ser cliente e/ou servidor
- Isto é, pode ser consumidor e/ou provedor de recursos

- Exemplo: sistemas para compartilhamento de arquivos na Internet (exemplo Torrent)

Slide 17- Verificação e Validação

Obejtivo

- Assegurar que o software
 - Cumpra suas especificações
 - Atenda às necessidades dos usuários e clientes

Verificação (Lógica): Verifica se o software está sendo construído conforme especificado.

Validação (Funcional): Verifica se o software atende às necessidades dos usuários.

- É importante corrigir defeitos nas fases iniciais por é mais barato
- Testes consomem cerca de 40% do tempo e custo de desenvolvimento

Princípios do Teste:

- Relacionar testes aos requisitos do cliente.
- Planejar testes antes do desenvolvimento.
- Teste completo é impossível.
- Aplicar o princípio de Pareto: 80% dos problemas vêm de 20% do código.
- Testes são mais eficazes quando realizados por terceiros.

Definições e Objetivos:

- Executar/observar um programa para revelar erros.
- Aumentar a confiança de que o software funciona conforme especificado.

Defeito: Instrução ou comando incorreto

Erro: Desvio da especificação

Falha: Processamento incorreto e comportamento inconsistente

Origem dos Defeitos:

- Maioria é de origem humana.
- Comuns em requisitos (56%), análise e design (27%), implementação (7%).

Fases de teste

1. Teste de Unicidade: Identifica erros em cada módulo separadamente
2. Teste de Integração: Verifica erros nas interfaces entremódulos
3. Teste de Sistema: Confirma que todas as funções estão conforme especificado e se integram corretamente
4. Teste de Aceitação:
 - a. Teste Alfa: Realizado pelo cliente em ambiente controlado pelo desenvolvedor
 - b. Teste Beta: Realizado nas instalações do cliente sem a presença do desenvolvedor, pelo usuário final do software

Casos de Teste

COMPONENTES

- Dados de Teste (Entrada): Necessário para executar o produto em teste
- Saída: Resultado esperado conforme especificação
- Oráculo: Mecanismo que define a saída esperada

TECNICAS DE TESTE

1. Teste de Caixa Preta:
 - Baseado na especificação, sem conhecimento dos detalhes de implementação
 - Foca no software do ponto de vista do usuário
2. Teste da Caixa Branca:
 - Baseado na lógica interna do código
 - Testa detalhes procedimentais do software
3. Outras abordagens:

- Teste de Performance, Usabilidade, Carga, Documentação, Sistemas de Tempo Real, Operação, Regressão.

Slide 19- Cobertura para testes unitários

Complexidade Ciclomática

1. Forma de calcular $V(G) = R$

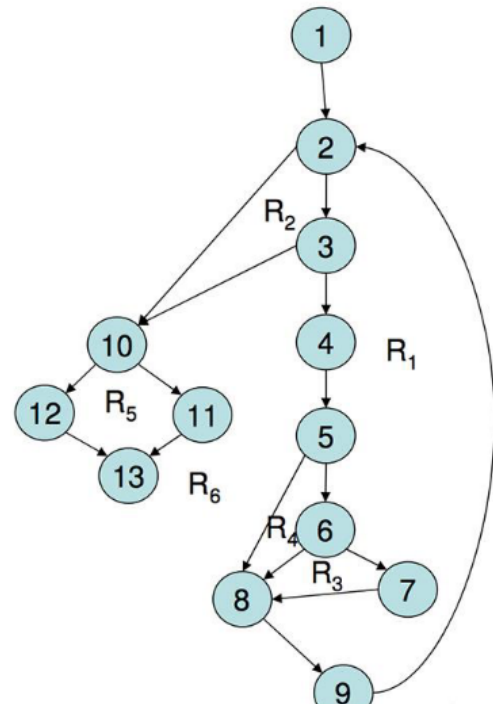
- Onde R é o número de regiões do grafo de fluxo
- $V(G) = 6$ regiões

2. Forma de calcular $V(G) = E - N + 2$

- Onde E é o número de arestas (setas) e N é o número de nós do grafo G
- $V(G) = 17 - 13 + 2 = 6$

3. Forma de calcular $V(G) = P + 1$

- Onde P é o número de nós-Predicado do grafo
- $V(G) = 5$ nós-predicado + 1 = 6



- Nós-predicado são aqueles que podem desviar o fluxo da execução: if, while, switch etc.

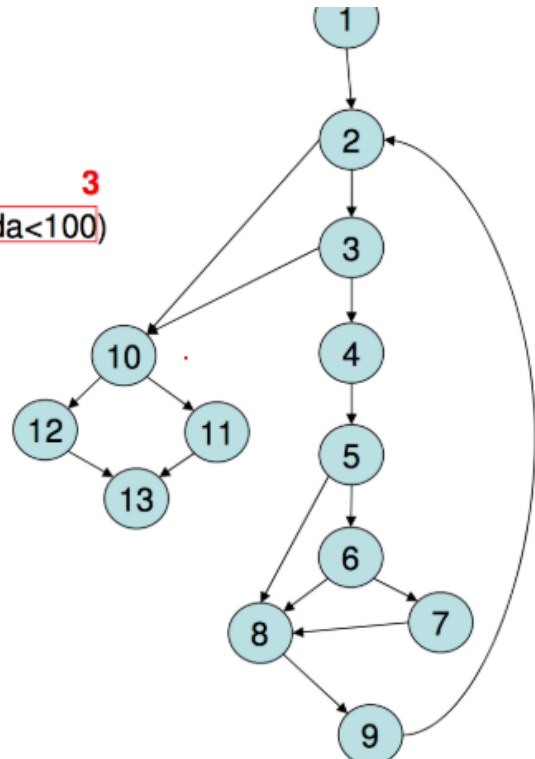
Os 6 caminhos independentes são:

1. 1-2-10-12-13
2. 1-2-10-11-13
3. 1-2-3-10-11-13
4. 1-2-3-4-5-8-9-2-[...]
5. 1-2-3-4-5-6-8-9-2-[...]
6. 1-2-3-4-5-6-7-8-9-2-[...]

Procedimento média(valor[])

```

1 i=1;
soma=0;
total.entrada=0;
total.válidas=0
2 Faça-Enquanto (valor[i]≠-999 E total.entrada<100)
3
4     incremente total.entrada de 1;
5     Se (valor[i]≥min E valor[i]≤max)
6
7         Então
8             incremente total.válidas de 1;
9             soma = soma + valor[i]
10         Fim-Se
11     incremente i de 1;
12 Fim-Enquanto
13 Se total.válidas>0
14     Então média = soma/total.válidas;
15 Senão média = -999;
16 Fim-Se
17 Fim média
    
```

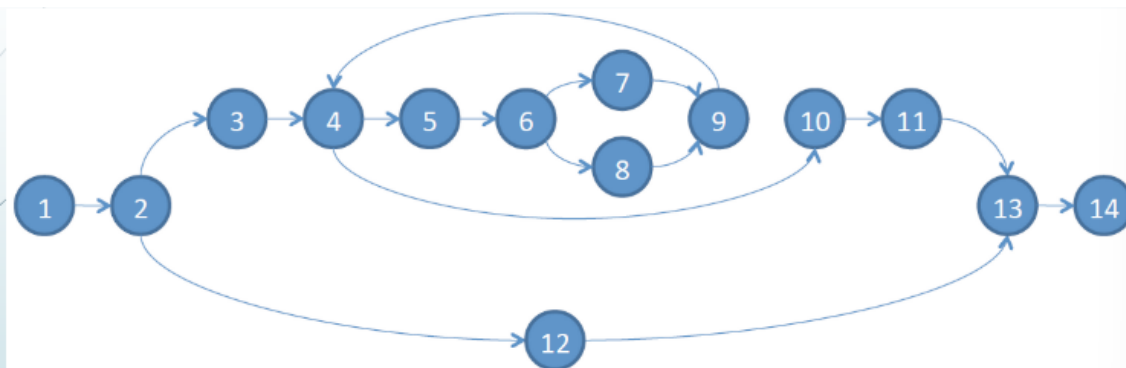


Complexidade Ciclomática

1. Gerar o gráfico de fluxo para cada método/procedimento/função
2. Calcular a complexidade ciclomática do grafo de fluxo
3. Identificar os caminhos independentes
4. Elaborar casos de teste com valores de entrada que exercitem cada um dos caminhos independentes

```

public List<String> ordena(List<String> listaDesordenada) {
1  List<String> listaOrdenada = new ArrayList<String>();
2  if (listaDesordenada.size() > 1) {
3      String pivo = listaDesordenada.get(0);
4      List<String> listaMenoresDesordenada = new ArrayList<String>();
5      List<String> listaMaioresDesordenada = new ArrayList<String>();
6      for (int i = 1; i < listaDesordenada.size(); i++) {
7          String elemento = listaDesordenada.get(i);
8          if (elemento.compareTo(pivo) < 0) { listaMenoresDesordenada.add(elemento); }
9          else { listaMaioresDesordenada.add(elemento); }
10     }
11     listaOrdenada.addAll(ordena(listaMenoresDesordenada));
12     listaOrdenada.add(pivo);
13     listaOrdenada.addAll(ordena(listaMaioresDesordenada));
14 } else { listaOrdenada.addAll(listaDesordenada); }
15 return listaOrdenada;
16 }
    
```

- $V(G) = E - N + 2 = 16 - 14 + 2 = 4$
 - Teste 1: 1, 2, 12, 13, 14
 - Teste 2: 1, 2, 3, 4, 10, 11, 13, 14 (IMPOSSÍVEL)
 - Teste 3: 1, 2, 3, 4, 5, 6, 7, 9, 4, 10, 11, 13, 14
 - Teste 4: 1, 2, 3, 4, 5, 6, 8, 9, 4, 10, 11, 13, 14

Slide 20- Test Driven Development (TDD)

Testes de software mostram a presença de bugs, mas não a sua ausência.

- Prática de programação proposta pelo XP
- Enfatiza a criação de testes automatizados antes da implementação do código

Benefícios

- Evita que os desenvolvedores esqueçam dos testes
- Favorece a escrita de código com testabilidade
- Cobertura dos testes tende a ser maior
- Favorece ao Clean Code- o desenvolvedor é o primeiro cliente do seu próprio código
- Podemos ter 3 estados neste processo
 - Teste com falha
 - Teste ok
 - Refatoração

Slide 21- Inspeção de Software

Definição

- Método de análise estática para verificar propriedades de qualidade de produtos de software

Características

- Processo estruturado e bem definido
- Equipe de inspeção é geralmente construída de pessoal técnico
- Participantes do processo têm papéis bem definidos

Inspeção → Requisitos → Projeto → Protótipos

Testes → Protótipos → Programas

Benefícios

- Menor custo para correção de defeitos
- Melhoram a produtividade
- “Testes podem somente revelar a presença de defeitos, não a ausência”

Taxonomia de Defeitos

- Omissão (O): qualquer informação necessária que tenha sido omitida.
- Fato Incorreto (FI): informação que consta do artefato mas que seja contraditória com o conhecimento que se tem do domínio de aplicação.
- Inconsistência (I): informação que consta do artefato mais de uma vez e em cada ocorrência ela é descrita de forma diferente.
- Ambiguidade (A): quando a informação pode levar a múltiplas interpretações.
- Informação Estranha (IE): qualquer informação que, embora relacionada ao domínio, não é necessária para o sistema em questão.
- Diversos (D): qualquer outro tipo de defeito que não se encaixe nas outras categorias.

Exemplos: Sistema para uma biblioteca

- Omissão:
 - Uma mensagem de alerta deve ser exibida quando o usuários tentar inserir um item incompleto
- Fato incorreto:
 - Um requisito diz "O sistema não deve aceitar devolução se o usuário não informar login e senha de seu cadastro"
 - Outro requisito diz "Para a devolução de livros não é necessário que o usuário informe quaisquer dados de seu cadastro, pois a baixa do livro será feita exclusivamente pelo código de barras do livro"
- Inconsistência:
 - Em um requisito: O sistema não deve permitir períodos de empréstimo superiores a 15 dias
 - Em outro: Os professores podem emprestar livros por um período de 3 semanas.
- Ambiguidade:
 - "Se o número de dias que o usuário está em atraso é menor que uma semana, ele deve pagar uma taxa de R\$1,00; se o número é maior que uma semana, a taxa é de R\$0,50 por dia."
 - Qual a taxa que deve ser paga se o período for exatamente uma semana?
- Informação estranha:
 - "quando um novo livro é adicionado ao acervo, ele permanece em uma prateleira especial por um período de um mês" • Esta informação não é necessária para o sistema.

Técnicas de leitura para inspeção

- Como detectar defeitos?
 - Analisar o documento
 - Entendendo o que o documento descreve em cada detalhe
 - Verificando as propriedades de qualidade requeridas

Métodologias de inspeção

INSPEÇÃO WALKTHROUGH

- Forma de avaliação de produto por especialistas que analisam previamente e discutem em reunião, sugerindo melhorias
 - Cada especialista propõe modificações nos documentos
 - Discussão em reunião com base no critério de cada um
 - Documentos modificados passam por um no walkthrough se necessário

Papéis

- Apresentador: Autor do artefato, explica e justifica.
- Coordenador: Moderador da reunião, mantém o foco.
- Secretário: Registra discussões e decisões.
- Oráculo de manutenção: Garante a compreensibilidade e manutenibilidade do artefato.
- Guardião dos padrões: Verifica a conformidade com os padrões da empresa.
- Representante do usuário: Assegura que o produto atenda às expectativas do cliente.
- Outros desenvolvedores: Contribuem com suas perspectivas.

INSPEÇÃO FANGAN

- Processo estruturado para encontrar defeitos em código, diagramas ou especificações
 - Artefatos de entrada e saída equivalem aos critérios de entrada e saída para as inspeções e qualquer desvio encontrado nos artefatos de saída é considerado um defeito
 - Defeitos são classificados como graves (não funcionamento) ou triviais (erro não funcional)

Papéis

- Autor: Criador do artefato.
- Narrador: Analisa e resume o artefato e critérios de aceitação.
- Revisores: Detectam defeitos no artefato.
- Moderador: Conduz a sessão de inspeção.

Atividades

- Planejamento: Preparação de materiais, convites e espaço.

- Visão geral: Instrução prévia e atribuição de papéis.
- Preparação: Análise dos artefatos pelos participantes, anotando defeitos e questões.
- Reunião de inspeção: Discussão e decisão sobre defeitos.
- Retrabalho: Correção dos defeitos pelo autor do artefato.
- Prosseguimento: Verificação de correções e aprovação para a próxima fase.

Slide 22- CMMI e MPS-BR

Definição geral de processo

- **Processo:** Conjunto de atividades realizadas com um determinado propósito, incluindo objetivos, métodos, guias, pessoas, produtos, entradas e saídas.
- Importância de um equilíbrio entre **pessoas, tecnologia, e processo**.
- A qualidade do produto de software está intimamente ligada à qualidade do processo de desenvolvimento, alcançada mais facilmente com base em modelos de maturidade como CMMI e MPS-BR

Modelos de melhoria de processos

CMMI (Capability Maturity Model Integration)

- Desenvolvido para **estabelecer melhores práticas** com base em estudos e conhecimento operacional.
- Dois tipos de modelos: contínuo e em estágios.

Objetivos:

- Aumento do foco das atividades
- Integração dos processos existentes
- Eliminar inconsistências
- Reduzir duplicações
- Fornecer terminologia comum
- Assegurar consistência com a norma ISO 15504;
- Flexibilidade

O objetivo do CMMI é prover um guia para a melhoria de processos organizacionais e habilidades de gestão do desenvolvimento, aquisição e manutenção de produtos e serviços.

- Auxilia a organização a avaliar a eficiência dos seus processos
- Estabelece prioridades de melhoria
- Auxilia a implementação das melhorias

- Representações: contínua (flexível, focada em áreas específicas) e por estágios (predefinido, focado na organização).

Slide 23- Refatoração

Tipos de manutenção

- **Preventiva:** correção de bugs óbvios.
- **Corretiva:** correção de bugs reportados pelos usuários.
- **Adaptativa:** customizações e atualizações de sistemas associados.
- **Evolutiva:** adição de novas funcionalidades.
- **Refatoração:** melhorias no código ou design sem afetar o comportamento externo.

Leis de Lehman

- Sistemas devem ser mantidos para serem úteis, mas manutenções aumentam a complexidade e deterioram a qualidade do código, a menos que a refatoração seja realizada para evitar isso.

Catálogo de refatoração

- Inclui técnicas como Extração de Métodos, Inline de Métodos, Movimentação de Métodos, Extração de Classes, Renomeação, entre outros.

Prática de refatoração

- Pode ser oportunista (realizada durante outras tarefas de programação) ou planejada (correção de problemas complexos).

Refatoração automatizada

- Realizada com o auxílio de uma IDE (Ambiente de Desenvolvimento Integrado).

Code Smells

- Indicadores de baixa qualidade de código, como:
 - Código Duplicado
 - Feature Envy (usa mais métodos e dados de outra classe, candidato a ser movido pra ela)
 - Métodos Longos
 - Classes Grandes
 - Métodos com Muitos Parâmetros
 - Variáveis Globais
 - Obsessão por Tipos Primitivos
 - Documentação

Débito Técnico

- Metáfora que destaca a importância de boas práticas de Engenharia de Software e os impactos negativos de soluções não otimizadas.

Slide 24- DevOps

- O termo "DevOps" refere-se à unificação das culturas de Desenvolvimento (Dev) e Operação (Ops) em torno da implantação de sistemas de software.
- O termo "DevOps" refere-se à unificação das culturas de Desenvolvimento (Dev) e Operação (Ops) em torno da implantação de sistemas de software.
- O objetivo é realizar a implantação de sistemas de forma suave e eficaz, eliminando o "empurra-empurra" entre Dev e Ops.
- Princípios de DevOps incluem a adoção de práticas ágeis na fase de implantação, transformando implantações em eventos não traumáticos e automatizando o processo de

implantação.

Práticas de DevOps

Controle de Versões:

- Fundamental para o desenvolvimento colaborativo, armazenando a versão mais atual do sistema e permitindo recuperar versões anteriores.

Integração Contínua (CI):

- Integração frequente do código, geralmente pelo menos uma vez por dia, com build e testes automatizados.

Deployment Contínuo (CD):

- Implantação imediata de commits/merges em produção, visando experimentação e feedback rápidos.

-
- O desenvolvimento baseado no Branch Principal (Trunk Based Development) promove o desenvolvimento direto no branch principal para minimizar conflitos e facilitar a identificação precoce de problemas de integração.
 - Feature Flags/Toggles são usadas para habilitar ou desabilitar funcionalidades em desenvolvimento, evitando que implementações parciais cheguem aos usuários finais.